



UNIVERSIDAD

Taller de integración y diferenciación

Estudiantes

Lenin Enrique García Caraballo

David Orlando Riaño Morales

Docente

Alexander Pérez Ruiz

Escuela Colombiana de Ingeniería Julio Garavito

Métodos computacionales en ingeniería – MCEI

2026-2

27 de septiembre de 2026

1 Ejercicio 1

1.1 Análisis matemático

1. Determine si puede obtenerse una expresión analítica y, si la obtiene, úsela como referencia; de lo contrario, defina una referencia numérica de alta precisión.

Se tiene la integral:

$$I = \int e^{-0,4x} (1 + 0,5 \sin(3x)) dx$$

Se distribuye la integral:

$$I = \int e^{-0,4x} dx + 0,5 \int e^{-0,4x} \sin(3x) dx$$

Para la integral de la izquierda:

$$\int e^{-0,4x} dx = \frac{1}{-0,4} e^{-0,4x} = -2,5 e^{-0,4x}$$

Para la integral de la derecha, se puede usar la fórmula de integración por partes sucesivas:

$$\int e^{ax} \sin(bx) dx = \frac{e^{ax}}{a^2 + b^2} (a \sin(bx) - b \cos(bx))$$

En este caso $a = -0,4$ y $b = 3$. El denominador es $a^2 + b^2 = (-0,4)^2 + (3)^2 = 9,16$. Sustituyendo en la fórmula:

$$\int e^{-0,4x} \sin(3x) dx = \frac{e^{-0,4x}}{9,16} (-0,4 \sin(3x) - 3 \cos(3x))$$

Reemplazando ambos resultados en la ecuación original y multiplicando la segunda integral por 0,5:

$$I = -2,5 e^{-0,4x} + 2,5 (-0,32751 \sin(3x) - 0,04366 \cos(3x)) + C$$

Al evaluar la integral en el intervalo $[0, 8]$, el resultado es:

$$I = 2,559824508$$

2. Explique qué representa el área acumulada.

El área acumulada bajo la curva representa la suma de la señal a lo largo de la variable x .

3. Identifique características que puedan afectar la aproximación.

La función dada es oscilatoria. Si se emplean métodos como la regla del trapecio o de Simpson, con un paso de discretización muy grande, el método podría no capturar correctamente los picos y valles de la oscilación, aumentando el error.

1.2 Resultados de GNU Octave

Implementando de forma manual la regla compuesta del trapecio en el código **punto_1_manual.m**, se realizaron seis ejecuciones del código variando la cantidad de trapecios $n = \{10, 20, 50, 100, 500, 1000\}$ por cada ejecución, con un intervalo de integración de 0 a 8. De esta forma pudo evaluarse el resultado obtenido a medida que se aumentaban los trapecios, el error absoluto y el error relativo entre el resultado de referencia obtenido analíticamente (2.55982) y el resultado obtenido para cada cantidad de trapecios, como también el tiempo de ejecución.

Cuadro 1: Trapecio Compuesto Implementado (Manual)

n	Integral Aproximada	Error Absoluto	Error Relativo	Tiempo (s)
10	2,506750	$5,306955 \times 10^{-2}$	0,02073175	0,000571
20	2,546354	$1,34661 \times 10^{-2}$	0,00526056	0,000154
50	2,557593	$2,26642 \times 10^{-3}$	0,00086984	0,000210
100	2,559258	$5,62384 \times 10^{-4}$	0,00021970	0,000188
500	2,559801	$1,85044 \times 10^{-5}$	0,00000723	0,000229
1000	2,559819	$1,25616 \times 10^{-6}$	0,00000049	0,000357

```
>> punto_1
resultado = 2.4941
Para n = 11:
Integral aproximada: 2.494073
Error absoluto: 6.574749e-02
Error relativo: 2.568442 %
Tiempo: 0.000723 segundos
>> punto_1
resultado = 2.5449
Para n = 21:
Integral aproximada: 2.544930
Error absoluto: 1.489028e-02
Error relativo: 0.581693 %
Tiempo: 0.001016 segundos
>> punto_1
resultado = 2.5575
Para n = 51:
Integral aproximada: 2.557503
Error absoluto: 2.317223e-03
Error relativo: 0.090523 %
Tiempo: 0.000476 segundos
>> punto_1
resultado = 2.5592
Para n = 101:
Integral aproximada: 2.559246
Error absoluto: 5.737920e-04
Error relativo: 0.022415 %
Tiempo: 0.000554 segundos
>> punto_1
resultado = 2.5598
Para n = 501:
Integral aproximada: 2.559801
Error absoluto: 1.85054e-05
Error relativo: 0.000726 %
Tiempo: 0.000484 segundos
>> punto_1
resultado = 2.5598
Para n = 1001:
Integral aproximada: 2.559819
Error absoluto: 1.267697e-06
Error relativo: 0.000050 %
Tiempo: 0.000621 segundos
>> |
```

Figura 1: Resultados de la regla compuesta de los trapecios en Octave

Posteriormente, se realizó el código **punto_1_trapz.m** implementando el método de los trapecios nativo de Octave **trapz**. En este caso, también se midió el resultado de la integral tomando las mismas cantidades de trapecios y el mismo intervalo de integración de 0 a 8. Además, se calcularon los respectivos errores absolutos y relativos entre el valor de referencia y el valor calculado, junto con los tiempos de ejecución, igual a como se registró en el cuadro 1.

Cuadro 2: Con función trapz de Octave

n	Integral Aproximada	Error Absoluto	Error Relativo	Tiempo (s)
10	2,494073	$6,5747 \times 10^{-2}$	2,568442 %	0,000723
20	2,544930	$1,48902 \times 10^{-2}$	0,581693 %	0,001016
50	2,557503	$2,31722 \times 10^{-3}$	0,090523 %	0,000476
100	2,559246	$5,7379 \times 10^{-4}$	0,022415 %	0,000554
500	2,559801	$1,8597 \times 10^{-5}$	0,000726 %	0,000484
1000	2,559819	$1,2677 \times 10^{-6}$	0,000050 %	0,000621

```

>> punto_1_OtroMetodo

Para n = 11:
Integral por regla compuesta (manual): 2.506750
Error absoluto: 5.306955e-02
Error relativo: 2.073175 %
Tiempo: 0.000571 segundos
>> punto_1_OtroMetodo

Para n = 21:
Integral por regla compuesta (manual): 2.546354
Error absoluto: 1.346610e-02
Error relativo: 0.526056 %
Tiempo: 0.000154 segundos
>> punto_1_OtroMetodo

Para n = 51:
Integral por regla compuesta (manual): 2.557593
Error absoluto: 2.226642e-03
Error relativo: 0.086984 %
Tiempo: 0.000210 segundos
>> punto_1_OtroMetodo

Para n = 101:
Integral por regla compuesta (manual): 2.559258
Error absoluto: 5.623835e-04
Error relativo: 0.021970 %
Tiempo: 0.000188 segundos
>> punto_1_OtroMetodo

Para n = 501:
Integral por regla compuesta (manual): 2.559801
Error absoluto: 1.850439e-05
Error relativo: 0.000723 %
Tiempo: 0.000229 segundos
>> punto_1_OtroMetodo

Para n = 1001:
Integral por regla compuesta (manual): 2.559819
Error absoluto: 1.256162e-06
Error relativo: 0.000049 %
Tiempo: 0.000357 segundos
>> |

```

Figura 2: Resultados usando la función trapz de Octave

1.3 Resultados de C++ con GSL

```

Resultados Cuadratura Adaptativa (GSL)
Resultado : 2.559824508
Error estimado por GSL : 2.841976108e-14
Error exacto (vs Ref) : 3.020623751e-10
Tolerancia absoluta : 1e-06
Tolerancia relativa : 1e-06

Resultados de método del trapecio)
Resultado (n=500) : 2.559801403
Error exacto (vs Ref) : 2.310454071e-05

```

Figura 3: Resultados en C++ con GSL

Para el código **punto_1.cpp** los resultados muestran que la cuadratura adaptativa de GSL, basada en la regla de Gauss-Kronrod (en los comentarios del código se explica a profundidad), obtiene una aproximación muy precisa de la integral. Para este caso se establecieron una tolerancia absoluta y una tolerancia

relativa de $1 * 10^{-6}$. El error respecto al valor de referencia fue $3,02 * 10^{-10}$, prácticamente el error fue de cero. Además, GSL calculó un error estimado de $2,84 * 10^{-14}$, el cual es pequeño.

La regla compuesta del trapecio utilizando $n = 500$ subintervalos obtuvo un error de $2,31 * 10^{-5}$. Aunque este resultado también es una buena aproximación, es menos preciso que GSL. Esto ocurre, porque el método del trapecio utiliza subintervalos de igual tamaño en todo el intervalo.

La diferencia entre ambos métodos está en la forma en que distribuyen los puntos donde evalúan la función. El método del trapecio calcula puntos con el mismo paso en todo el intervalo. La cuadratura adaptativa puede tomar más puntos en las zonas donde la función presenta mayores cambios y utilizar menos evaluaciones en las zonas donde cambia poco.

Explique qué queda bajo control del programador y qué delega GSL

Al utilizar GSL desde C/C++, el programador no necesita implementar manualmente los pasos del método de integración. Su función es definir el problema y establecer las condiciones bajo las cuales se realizará el cálculo. Se debe dar la función que se desea integrar, indicar los límites del intervalo y configurar parámetros del método. También es necesario reservar el espacio de trabajo que utilizará GSL y establecer el número máximo de subdivisiones permitidas.

Además, el programador puede definir el nivel de precisión que desea obtener mediante las tolerancias absoluta y relativa. En este caso, también se selecciona la regla de Gauss-Kronrod que utilizará GSL para realizar la integración.

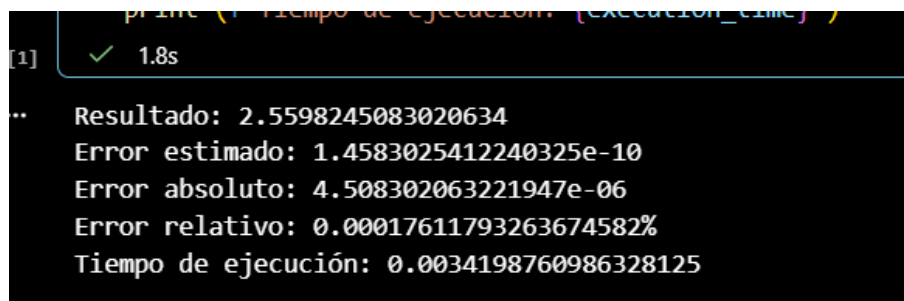
Una vez configurados estos parámetros, GSL se encarga de realizar el proceso de integración. La cuadratura adaptativa de GSL puede dividir el intervalo de manera automática cuando necesita obtener mayor precisión.

GSL también calcula una estimación del error de la integración. Esta información permite al algoritmo determinar si el resultado cumple con las tolerancias establecidas y continuar realizando subdivisiones cuando sea necesario.

1.4 Resultados de Python con SciPy

Al implementar la integral con Python empleando la librería SciPy en el cuadernillo `python_integracion_diferenciacion` se utilizó la función `quad`, la cual requiere como argumento la expresión a integrar definida como una función de Python, y el intervalo de integración desde un punto $a = 0$ a $b = 8$. No se requiere definir la cantidad de unidades en que se va a dividir el intervalo de integración para sumar sus respectivas áreas, como ocurre en otros métodos en otros lenguajes (como por ejemplo en Octave o en C++ a la hora de usar el método de los trapecios).

Los resultados obtenidos con este código fueron los siguientes:



```
print('Tiempo de ejecución: {execution_time}')
```

```
[1] ✓ 1.8s
```

```
.. Resultado: 2.5598245083020634
   Error estimado: 1.4583025412240325e-10
   Error absoluto: 4.508302063221947e-06
   Error relativo: 0.00017611793263674582%
   Tiempo de ejecución: 0.0034198760986328125
```

Figura 4: Resultados del código de Python usando SciPy

En Python, con la librería SciPy, el método de cuadratura adaptativa es mucho más simple de aplicar en cuanto a sintaxis debido a que requiere muchos menos parámetros. Las razones son las siguientes:

- En C++ con GSL, exige reservar memoria de forma manual en el sistema antes de iniciar a calcular la integral con `gsl_integration_workspace_alloc`, para luego liberar la memoria con `gsl_integration_workspace_free`. Además, que C++ con GSL exige usar punteros porque la función matemática no puede pasarse de forma directa a una estructura `gsl_function`.
- SciPy administra la memoria, el espacio de trabajo y las subdivisiones de forma automática por debajo de la interfaz. Por lo que la función `quad` acepta de forma directa la función a integrar definida como un lambda.

1.5 Preguntas de análisis

1. ¿Qué cambia en el trapecio al aumentar n ?

Al aumentar la cantidad de trapecios n , el ancho del trapecio va a disminuir. Si n es muy grande, el ancho del trapecio llegará a ser infinitesimal. Esto teniendo en cuenta la ecuación de su ancho.

$$h = \frac{a - b}{n} \quad (1)$$

Además, al aumentar n , al haber más trapecios, la suma de todas sus areas va a converger o va a ser más cercana al valor original de la integral indefinida.

2. ¿Qué diferencia hay entre aumentar n y reducir la tolerancia de un método adaptativo?

Al aumentar n , nos estamos asegurando ciegamente de acercarnos más al valor real de la integral definida. Simplemente aumentar n nos va a asegurar que vamos a estar más cerca del valor deseado, pero no que tan cerca vamos a estar. Además que se desperdician recursos computacionales evaluando zonas de la función que no lo necesitan.

En cambio, al reducir la tolerancia, estamos limitando hasta un valor conocido cercano al valor real de la integral definida. Por ejemplo: Si la tolerancia absoluta del método es $epsabs = 1 \cdot 10^{-6}$, el método no se va a detener hasta que el resultado sea diferente en un valor de $1 \cdot 10^{-6}$ al valor deseado. Por lo que es un parametro que nos permite definir que tan cercanos queremos estar al valor deseado.

3. ¿Mayor precisión implica necesariamente mayor costo?

Una mayor precisión no implica necesariamente un mayor costo computacional, ya que la precisión puede depender de la eficiencia del algoritmo. Hay métodos que le exigen menos al computador, y que pueden cubrir mejor el área bajo la curva de la función.

Por ejemplo, para obtener una ganancia significativa en precisión, el método nos obliga a aumentar n drásticamente. Esto incrementa el costo computacional de forma bruta e ineficiente, evaluando masivamente zonas que no lo necesitan.

En cambio, métodos como el de cuadratura adaptativa alcanzan una precisión superior con muchas menos evaluaciones en total. Logran esto optimizando las ubicaciones de los puntos de muestreo y distribuyendo el costo de cálculo únicamente en las regiones complejas de la función.

4. Compare el error estimado por GSL/SciPy con el error respecto a la referencia.

El error estimado por GSL o SciPy es simplemente un error interno del método. Es decir, el algoritmo compara internamente el resultado de evaluar la función con una resolución baja contra una resolución alta, y nos dice qué tanta diferencia hay. Por eso nos arroja valores tan pequeños como $2,84e - 14$ en C++ o $1,45e - 10$ en Python.

En cambio, el error respecto a la referencia es la diferencia exacta contra el valor analítico real que nosotros mismos definimos en el código. Al ver los resultados, notamos que este error nos dio mucho más alto (por ejemplo, $4,508302e - 06$ en Python). Esto pasó porque nosotros truncamos el valor de la referencia a solo unos pocos decimales ($2,55982$). Por lo que el método numérico adaptativo terminó calculando con mayor precisión que el propio valor de referencia que le dimos para compararse.

2 Ejercicio 2

2.1 Resultados de GNU Octave

A continuación, se muestran los resultados del código `punto_2.m`:

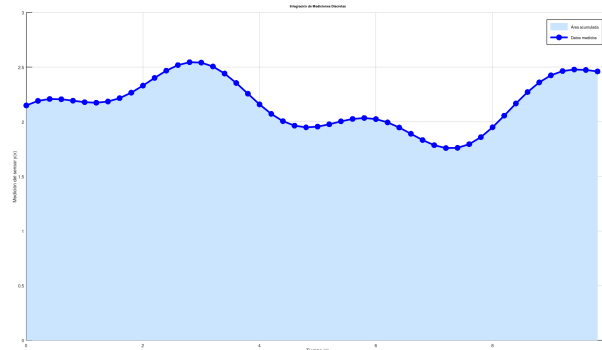


Figura 5: Resultados en GNU Octave

¿Cómo se soluciona el problema tomando en cuenta las condiciones para aplicar Simpson?

La indicación del taller de aplicar la regla de Simpson cuando se cumplan sus condiciones, es porque, para utilizar la regla de Simpson 1/3 compuesta, el número de subintervalos debe ser par. Esto ocurre porque el método utiliza grupos de tres puntos para aproximar la función con una parábola.

En este caso, hay 50 mediciones. Como cada par de puntos consecutivos forma un subintervalo, 50 puntos generan 49 subintervalos. Debido a que 49 es un número impar, no es posible aplicar la regla de Simpson a todos los datos.

Para solucionarlo, se utiliza la regla de Simpson en los primeros 49 puntos. Estos puntos forman 48 subintervalos, y como 48 es un número par, en esta parte de los datos sí se cumple la condición necesaria para aplicar Simpson.

Después de aplicar Simpson a los primeros 49 puntos, queda un último punto que forma un único subintervalo con el punto anterior. Para ese último intervalo no hay suficientes puntos para formar otro grupo de tres puntos y aplicar Simpson. Por esta razón, se utiliza la regla del trapecio para calcular el área de ese último intervalo.

Por último, se suman los dos resultados (la aproximación obtenida mediante Simpson para los primeros 48 subintervalos y la aproximación mediante trapecio para el último subintervalo).

2.2 Resultados de C++ con GSL

Los resultados del código `punto_2.cpp` se muestran a continuación:

```
Cantidad de datos procesados: 50
Integral (Trapecio directo) : 21.190846
Integral (Spline GSL)      : 21.191888
```

Figura 6: Resultados en C++ con GSL

Al aplicar la regla del trapecio, se obtuvo un área de 21,190846. Una ventaja de este método es que es sencillo de implementar y requiere pocos cálculos. Además, trabaja directamente con los datos disponibles, sin necesidad de construir una función adicional entre los puntos.

Sin embargo, el método del trapecio supone que entre dos puntos consecutivos la función puede aproximarse con una línea recta, aumentando el error. Por ello, la precisión del método depende de qué tan

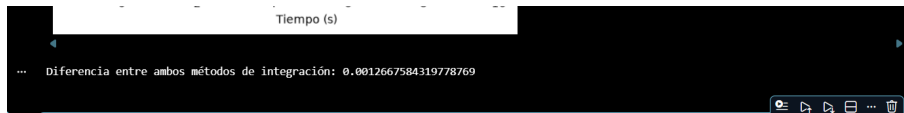


Figura 8: Diferencia entre regla del trapecio y Simpson

separados estén los datos y de qué tan bien representan el comportamiento de la señal.

Al utilizar GSL para construir un spline cúbico (el spline analiza el punto anterior y el punto siguiente para calcular con qué pendiente venía la curva, después, une los puntos con una curva suave) y posteriormente integrarlo, se obtuvo un área de 21,191888. En lugar de unir los puntos con líneas rectas, GSL construye una curva suave que pasa por las mediciones. Esto permite representar mejor la forma de la señal entre los puntos y realizar la integración sobre toda esa curva.

Una ventaja del spline es que permite tener una representación continua de los datos. Por ejemplo, aunque no exista una medición exactamente en una posición, la curva interpolada permite estimar el valor de la señal en ese punto y utilizarlo para calcular la integral.

Una desventaja, es que este método requiere más cálculos que la regla del trapecio, ya que primero debe construir la curva interpolada antes de integrarla. Además, si los datos contienen mucho ruido o cambios bruscos, una interpolación con splines puede producir comportamientos que no representan los datos originales.

En este caso, ambos métodos son muy cercanos, 21,190846 con trapecio y 21,191888 con spline. La diferencia es porque el trapecio aproxima la señal con líneas rectas, mientras que el spline utiliza curvas cúbicas para representar el comportamiento entre las mediciones.

2.3 Resultados de Python con SciPy

Se implementó un código en Python en el cuadernillo `python_integracion_diferenciacion.ipynb` que toma los datos del excel del sensor, y aplica la regla del trapecio para calcular el área bajo la curva estimada. Luego se hace lo mismo con el método de Simpson y se grafican los datos del excel.

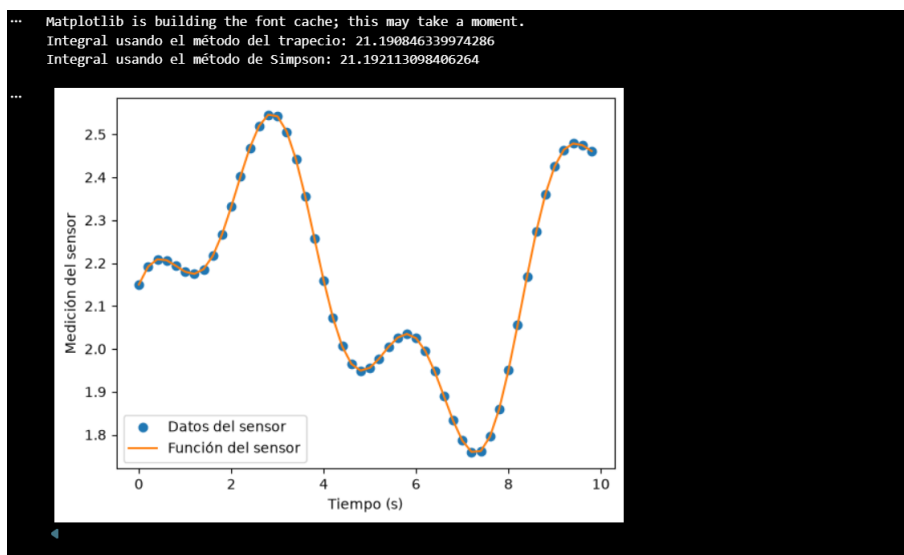


Figura 7: Resultados obtenidos con SciPy Python empleando el método del trapecio y Simpson

Al calcular la diferencia entre los valores obtenidos, se obtuvo lo siguiente:

Si se comparan los valores obtenidos, ambos tienden a converger a 21.19. Como puede verse, la diferencia entre ambos es de 0.001267, siendo el mayor de valor el obtenido con Simpson.

Teniendo en cuenta que solo se contaba con los datos medidos por el sensor y no con la función exacta, la diferencia podría no ser muy significativa debido a la cantidad de aproximaciones que se hacen. Pero si se tuviera que escoger el método más exacto, por geometría, debido a que este no utiliza rectas para aproximar los valores de la función, sino que utiliza parábolas o curvas suaves.

3 Extensión: diferenciación numérica

3.1 En GNU Octave

El código de Octave empleado para esta solución es **diferenciacion.m**, y sus resultados fueron:

i	x	y	dy (Adelante)	dy (Central)
0	0.000000	2.150000	0.209017	0.209017
1	0.200000	2.191803	0.085202	0.147110
2	0.400000	2.208844	-0.011276	0.036963
3	0.600000	2.206589	-0.065108	-0.038192
4	0.800000	2.193567	-0.069089	-0.067099
5	1.000000	2.179749	-0.025335	-0.047212
6	1.200000	2.174682	0.055147	0.014906
7	1.400000	2.185712	0.154434	0.104791
8	1.600000	2.216599	0.250881	0.202657
9	1.800000	2.266775	0.322968	0.286924
10	2.000000	2.331368	0.353136	0.338052
11	2.200000	2.401995	0.330935	0.342035
12	2.400000	2.468182	0.254945	0.292940
13	2.600000	2.519172	0.133124	0.194035
14	2.800000	2.545796	-0.018472	0.057326
15	3.000000	2.542102	-0.178380	-0.098426
16	3.200000	2.506426	-0.323363	-0.250872
17	3.400000	2.441753	-0.432365	-0.377864
18	3.600000	2.355281	-0.490082	-0.461223
19	3.800000	2.257264	-0.489558	-0.489820
20	4.000000	2.159353	-0.433332	-0.461445
21	4.200000	2.072686	-0.332945	-0.383138
22	4.400000	2.006097	-0.206905	-0.269925
23	4.600000	1.964716	-0.077447	-0.142176
24	4.800000	1.949227	0.033343	-0.022052
25	5.000000	1.955895	0.107349	0.070346
26	5.200000	1.977365	0.133561	0.120455
27	5.400000	2.004077	0.109949	0.121755
28	5.600000	2.026067	0.043764	0.076856
29	5.800000	2.034820	-0.049781	-0.003009
30	6.000000	2.024864	-0.150228	-0.100005
31	6.200000	1.994818	-0.235513	-0.192870
32	6.400000	1.947716	-0.285891	-0.260702
33	6.600000	1.890537	-0.287465	-0.286678
34	6.800000	1.833044	-0.234703	-0.261084
35	7.000000	1.786104	-0.131498	-0.183100
36	7.200000	1.759804	0.009399	-0.061049
37	7.400000	1.761684	0.168499	0.088949
38	7.600000	1.795384	0.322862	0.245680
39	7.800000	1.859956	0.449992	0.386427
40	8.000000	1.949955	0.531667	0.490830
41	8.200000	2.056288	0.557022	0.544344
42	8.400000	2.167693	0.524382	0.540702
43	8.600000	2.272569	0.441491	0.482937
44	8.800000	2.360867	0.324134	0.382813
45	9.000000	2.425694	0.193357	0.258746
46	9.200000	2.464366	0.071805	0.132581
47	9.400000	2.478727	-0.020220	0.025792
48	9.600000	2.474683	-0.068266	-0.044243
49	9.800000	2.461029	-0.068266	-0.068266

Figura 9: Resultados en GNU Octave

3.2 En C++ con GSL

El código en C++ empleado para esta solución es **diferenciacion.cpp**, y sus resultados fueron:

i	x	y	dy (Adelante)	dy (Central)
0	0.000000	2.150000	0.209017	0.209017
1	0.200000	2.191803	0.085202	0.147110
2	0.400000	2.208844	-0.011276	0.036963
3	0.600000	2.206589	-0.065108	-0.038192
4	0.800000	2.193567	-0.069089	-0.067099
5	1.000000	2.179749	-0.025335	-0.047212
6	1.200000	2.174682	0.055147	0.014906
7	1.400000	2.185712	0.154434	0.104791
8	1.600000	2.216599	0.250881	0.202657
9	1.800000	2.266775	0.322968	0.286924
10	2.000000	2.331368	0.353136	0.338052
11	2.200000	2.401995	0.330935	0.342035
12	2.400000	2.468182	0.254945	0.292940
13	2.600000	2.519172	0.133124	0.194035
14	2.800000	2.545796	-0.018472	0.057326
15	3.000000	2.542102	-0.178380	-0.098426
16	3.200000	2.506426	-0.323363	-0.250872
17	3.400000	2.441753	-0.432365	-0.377864
18	3.600000	2.355281	-0.490082	-0.461223
19	3.800000	2.257264	-0.489558	-0.489820
20	4.000000	2.159353	-0.433332	-0.461445
21	4.200000	2.072686	-0.332945	-0.383138
22	4.400000	2.006097	-0.206905	-0.269925
23	4.600000	1.964716	-0.077447	-0.142176
24	4.800000	1.949227	0.033343	-0.022052
25	5.000000	1.955895	0.107349	0.070346
26	5.200000	1.977365	0.133561	0.120455
27	5.400000	2.004077	0.109949	0.121755
28	5.600000	2.026067	0.043764	0.076856
29	5.800000	2.034820	-0.049781	-0.003009
30	6.000000	2.024864	-0.150228	-0.100005
31	6.200000	1.994818	-0.235513	-0.192870
32	6.400000	1.947716	-0.285891	-0.260702
33	6.600000	1.890537	-0.287465	-0.286678
34	6.800000	1.833044	-0.234703	-0.261084
35	7.000000	1.786104	-0.131498	-0.183100
36	7.200000	1.759804	0.009399	-0.061049
37	7.400000	1.761684	0.168499	0.088949
38	7.600000	1.795384	0.322862	0.245680
39	7.800000	1.859956	0.449992	0.386427
40	8.000000	1.949955	0.531667	0.490830
41	8.200000	2.056288	0.557022	0.544344
42	8.400000	2.167693	0.524382	0.540702
43	8.600000	2.272569	0.441491	0.482937
44	8.800000	2.360867	0.324134	0.382813
45	9.000000	2.425694	0.193357	0.258746
46	9.200000	2.464366	0.071805	0.132581
47	9.400000	2.478727	-0.020220	0.025792
48	9.600000	2.474683	-0.068266	-0.044243
49	9.800000	2.461029	-0.068266	-0.068266

Figura 10: Resultados en C++

3.3 En Python con SciPy

El código en Python empleado para esta solución está en el cuadernillo `python_integracion_diferenciacion.ipynb`, y sus resultados fueron:

i	x	y	dy (Adelante)	dy (Central)
0	0.000000	2.150000	0.209017	0.209017
1	0.200000	2.191803	0.085202	0.147110
2	0.400000	2.208844	-0.011276	0.036963
3	0.600000	2.206589	-0.065108	-0.038192
4	0.800000	2.193567	-0.069089	-0.067099
5	1.000000	2.179749	-0.025335	-0.047212
6	1.200000	2.174682	0.055147	0.014906
7	1.400000	2.185712	0.154434	0.104791
8	1.600000	2.216599	0.250881	0.202657
9	1.800000	2.266775	0.322968	0.286924
10	2.000000	2.331368	0.353136	0.338052
11	2.200000	2.401995	0.330935	0.342035
12	2.400000	2.468182	0.254945	0.292940
13	2.600000	2.519172	0.133124	0.194035
14	2.800000	2.545796	-0.018472	0.057326
15	3.000000	2.542102	-0.178380	-0.098426
16	3.200000	2.506426	-0.323363	-0.250872
17	3.400000	2.441753	-0.432365	-0.377864
18	3.600000	2.355281	-0.490082	-0.461223
19	3.800000	2.257264	-0.489558	-0.489820
20	4.000000	2.159353	-0.433332	-0.461445
21	4.200000	2.072686	-0.332945	-0.383138
22	4.400000	2.006097	-0.206905	-0.269925
23	4.600000	1.964716	-0.077447	-0.142176
24	4.800000	1.949227	0.033343	-0.022052
25	5.000000	1.955895	0.107349	0.070346
26	5.200000	1.977365	0.133561	0.120455
27	5.400000	2.004077	0.109949	0.121755
28	5.600000	2.026067	0.043764	0.076856
29	5.800000	2.034820	-0.049781	-0.003009
30	6.000000	2.024864	-0.150228	-0.100005
31	6.200000	1.994818	-0.235513	-0.192870
32	6.400000	1.947716	-0.285891	-0.260702
33	6.600000	1.890537	-0.287465	-0.286678
34	6.800000	1.833044	-0.234703	-0.261084
35	7.000000	1.786104	-0.131498	-0.183100
36	7.200000	1.759804	0.009399	-0.061049
37	7.400000	1.761684	0.168499	0.088949
38	7.600000	1.795384	0.322862	0.245680
39	7.800000	1.859956	0.449992	0.386427
40	8.000000	1.949955	0.531667	0.490830
41	8.200000	2.056288	0.557022	0.544344
42	8.400000	2.167693	0.524382	0.540702
43	8.600000	2.272569	0.441491	0.482937
44	8.800000	2.360867	0.324134	0.382813
45	9.000000	2.425694	0.193357	0.258746
46	9.200000	2.464366	0.071805	0.132581
47	9.400000	2.478727	-0.020220	0.025792
48	9.600000	2.474683	-0.068266	-0.044243
49	9.800000	2.461029	-0.068266	-0.068266

Figura 11: Resultados en Python usando SciPy

3.4 Análisis de las diferenciaciones numéricas:

La diferencia hacia adelante calcula la pendiente utilizando el punto actual y el siguiente. El resultado depende de lo que ocurre después del punto donde se está calculando la derivada. La diferencia central utiliza tanto el punto anterior como el siguiente. Al utilizar información de ambos lados, esta aproximación representa mejor el comportamiento de la señal alrededor del punto analizado.

La diferencia entre ambos métodos también está en su precisión. En este ejercicio, los datos tienen un paso constante de $h = 0,2$. La diferencia hacia adelante tiene un error que depende de h y la diferencia central tiene un error que depende de h^2 . Cuando el paso entre los datos es pequeño, la diferencia central da una aproximación más precisa de la derivada. Es por eso que hay diferencias entre estos métodos, especialmente en las zonas donde la señal cambia rápidamente.

Hay un caso especial en el primer y último dato. La diferencia hacia adelante necesita conocer el punto siguiente, por lo que puede utilizarse normalmente desde el primer punto hasta el penúltimo. En el último punto ya no existe un dato posterior, así que se debe emplear una diferencia hacia atrás, que utiliza el punto anterior.

La diferencia central necesita un punto a cada lado. Por lo tanto, no puede utilizarse ni en el primer ni en el último dato. Para solucionar esto, se debe emplear una diferencia hacia adelante en el primer punto y una diferencia hacia atrás en el último.

- ¿Por qué la diferenciación suele ser más sensible al ruido que la integración?

La diferenciación es más sensible al ruido porque calcula la pendiente de la señal a partir de las diferencias entre mediciones cercanas. Por ejemplo, en una diferencia se restan dos valores y el resultado se divide entre el paso h . Si una de las mediciones tiene error debido al ruido del sensor, ese error afecta la resta. Además, como h suele ser pequeño, al dividir el ruido entre este valor, el efecto aumenta.

Para la integración, en vez de calcular cuánto cambia la señal entre dos puntos, se van sumando áreas a lo largo del intervalo. Con el ruido, algunas mediciones pueden quedar un poco por encima del valor real y otras un poco por debajo. Al sumar todas estas áreas, parte de esos errores puede compensarse entre sí.

4 Comparación final

Criterio	Octave	C/C++ + GSL	Python + SciPy
Facilidad de implementación	Alta. Operaciones que simplifican ciclos y permiten soluciones compactas.	Baja. Requiere manejo de memoria, definición de punteros y bibliotecas externas.	Muy alta. Manejo avanzado de arreglos (NumPy) e interfaces de alto nivel (SciPy).
Control del algoritmo	Medio. Se controlan los bucles lógicos en implementaciones manuales, pero se restringe en funciones nativas.	Muy alto. Exige configurar tolerancias, espacios de trabajo y subdivisiones manualmente.	Medio. Permite configurar tolerancias, pero no se pueden configurar todos los parámetros de la librería.
Manejo de tolerancias	Manual en implementaciones propias.	El programador es quien define tolerancias absolutas y relativas.	Integrado. Pueden ajustarse tolerancias, pero SciPy define valores predeterminados seguros.
Integración de funciones	Soportada, aunque en el taller se priorizó la regla manual del trapecio y la nativa <code>trapz</code> .	Excelente. Cuadratura adaptativa (Gauss-Kronrod) optimizada para funciones analíticas continuas.	Excelente. <code>scipy.integrate.quad</code> resuelve integrales analíticas con alta precisión sin bucles.
Integración de datos	Directa. Función nativa <code>trapz</code> y arreglos. Requiere ajuste manual para Simpson.	Avanzada. Permite integración por trapecio explícito o construcción de funciones interpoladas (splines).	Directa. Se resuelve fácilmente con segmentación de arreglos (slicing) o funciones de NumPy.
Estimación del error	Por comparación directa (referencia menos aproximación).	Automática (estadística). GSL sobreestima su error internamente garantizando precisión.	Automática. La función <code>quad</code> entrega el error estimado integrado en la tupla de resultado.
Tiempo de ejecución	Rápido. Operaciones vectorizadas y funciones optimizadas.	Muy rápido. Código compilado de bajo nivel que ejecuta cálculos iterativos eficientemente.	Rápido. Se apoya en librerías en C, pero es lenguaje interpretado.
Visualización	Nativa y sencilla (<code>plot</code> , <code>area</code>), muy integrada en el entorno matemático.	Nula. C/C++ carece de herramientas nativas de visualización integradas.	Excelente a través de módulos externos adicionales (por ejemplo, Matplotlib).

Cuadro 3: Comparación final de la implementación en los tres entornos

5 Reflexión

1. A partir de lo desarrollado, explique cómo cambia su estrategia cuando pasa de conocer una función $f(x)$ a disponer únicamente de datos experimentales (x_i, y_i) . ¿Qué información deja de estar disponible y cómo afecta la selección del método?

Cuando se conoce una función, es posible calcular su valor en cualquier punto del intervalo. Cuando solo se tienen datos experimentales, únicamente conocemos los valores que fueron medidos. Por lo tanto, no sabemos qué ocurre entre una medición y otra ni tenemos un valor exacto con el cual comparar el resultado y calcular el error real.

Cuando tenemos una función, podemos utilizar métodos adaptativos que calculan nuevos puntos donde sea necesario para obtener mayor precisión (como la cuadratura adaptativa de GSL). Con datos experimentales, no se pueden obtener nuevos valores en puntos que no fueron medidos. En ese caso, se puede trabajar directamente con los datos utilizando métodos como el trapecio o Simpson, o construir primero una función aproximada usando interpolación, como el spline cúbico, y después integrarla.

2. Reconstruya el proceso seguido para resolver los dos problemas en Octave, C/C++-GSL y Python-SciPy. ¿Qué conceptos sobre discretización, error, tolerancia, representación de datos y abstracción de bibliotecas considera ahora fundamentales para implementar métodos numéricos de manera confiable?

En GNU Octave fue necesario programar manualmente parte de la lógica del método. Por ejemplo, al utilizar la regla de Simpson con los datos disponibles, fue necesario comprobar si el número de intervalos cumplía las condiciones del método y utilizar la regla del trapecio para el último intervalo cuando era necesario. En C++ con GSL, fue necesario configurar más detalles del proceso, como el espacio de trabajo, las tolerancias y la regla de Gauss-Kronrod utilizada para la integración adaptativa. En Python con SciPy permite realizar estos cálculos de una manera más sencilla con funciones como "simpsonz" "trapezoid", que reciben directamente los datos almacenados en arreglos de NumPy y realizan internamente gran parte del procedimiento.

Cuando se trabaja con datos experimentales, solo se conocen los puntos que fueron medidos, por lo que no se puede aumentar indefinidamente la cantidad de información simplemente haciendo el paso h más pequeño. Además, utilizar un paso demasiado pequeño en la diferenciación puede aumentar el efecto del ruido y de los errores presentes en las mediciones. No siempre utilizar más puntos o un paso más pequeño produce un resultado mejor. Es necesario tener en cuenta la calidad real de los datos.

Las tolerancias y el nivel de automatización de cada biblioteca determinan cuánto control tiene el programador sobre el cálculo. GSL permite configurar aspectos como la precisión y las condiciones para detener la integración, mientras que SciPy facilita la implementación al encargarse automáticamente de gran parte del procedimiento.